Lab 2 Reflection ReportName: Gaponenko Stanislav

Statelessness Analysis:

The /echo endpoint does not need to keep a connection open (unlike the TCP socket in Lab 1) because REST over HTTP is inherently stateless. Every HTTP request must contain all the information necessary for the server to understand and process it. If I send 10 consecutive /echo requests, the server processes each one independently. The server does not, and should not, "remember" previous requests, as maintaining state between calls violates the REST architectural constraints.

Reflection Questions :

1. How does a REST request fundamentally differ from the TCP session in Lab 1 in terms of connection duration? In Lab 1, the raw TCP connection was long-lived, meaning the connection remained open for multiple message exchanges between the client and server. In contrast, a REST request over HTTP typically follows a short-lived request-response cycle: the client opens a connection, sends a request, receives a response, and the connection is subsequently closed or returned to a pool.

2. Why does /echo use POST rather than GET? What would change if you used GET with parameters in the URL? /echo uses POST because it implies sending a payload (the JSON body) to the server for processing. If GET were used, the message data would have to be appended to the URL as query string parameters (e.g., /echo?message=hello). This is less secure for sensitive data, is subject to URL length limits enforced by browsers and servers, and violates REST semantics, as GET should strictly be used for retrieving resources without side effects.

3. What happens to the messages in the orders queue if you stop consumer.py for 30 seconds while producer.py keeps running? Test this and describe the result. Based on the testing performed, if consumer.py is stopped, producer.py continues to successfully send messages to the broker. Because the queue acts as an asynchronous buffer, the broker stores these incoming orders in its in-memory list. When consumer.py is restarted after 30 seconds, it immediately begins fetching the accumulated messages in strict FIFO order, ensuring zero data loss and demonstrating the decoupling of sender and receiver.

4. Give an example of a real situation where a synchronous REST call would be the better choice over a message queue, and vice versa.

- **Synchronous REST call:** Better for immediate, user-facing actions where the client must wait for a definitive outcome before proceeding. For example, processing a credit card payment or validating login credentials.
- **Message queue (Asynchronous):** Better for heavy background processing where the user does not need to wait. For example, sending bulk email newsletters, generating massive PDF reports, or processing uploaded video files.